IsoQuant: Hardware-Aligned SO(4) Isoclinic Rotations for LLM KV Cache Compression

Zhongping Ji

March 29th 2026

Abstract

Orthogonal feature decorrelation is a powerful primitive for low-bit online vector quantization, but the standard recipe of applying a dense random orthogonal matrix incurs prohibitive $O(d^2)$ compute and storage cost. Recent work such as RotorQuant replaces the global rotation with blockwise 3D Clifford rotors, substantially reducing complexity. However, the resulting 3D partition is awkward for modern hardware because neural features are usually powers of two in width, leading to tail handling, misaligned memory access, and limited local mixing capacity.

We propose **IsoQuant**, a new 4D block rotation framework based on quaternion algebra and the isoclinic decomposition of $SO(4)$. Each block of four coordinates is identified with a quaternion and rotated by a closed-form sandwich product $T(v) = q_L v \overline{q_R}$ parameterized by a pair of unit quaternions. This yields two practical variants: *IsoQuant-Full*, which uses the full six degrees of freedom of $SO(4)$, and *IsoQuant-Fast*, which restricts to a single isoclinic factor for lower overhead. At $d = 128$, IsoQuant-Full reduces forward rotation cost from roughly 2,408 FMAs in RotorQuant to 1,024, while IsoQuant-Fast further reduces it to 512. Across 18 fused CUDA benchmark settings covering $d \in \{128, 256, 512\}$, bit width $\{2, 3, 4\}$, and both FP16/FP32 execution, IsoQuant-Full and IsoQuant-Fast achieve mean kernel-level speedups of 4.83× and 5.08× over RotorQuant while maintaining comparable reconstruction MSE, with peak speedups above 6×. At present, however, our empirical validation is limited to the stage-1 quantize–dequantize path on synthetic normalized vectors; we have not yet validated end-to-end KV-cache quality on real model activations. Code: https://github.com/ParaMind2025/isoquant.

1 Introduction

KV-cache compression has emerged as a key systems bottleneck for long-context large language model inference. A core insight behind online vector quantization methods such as TurboQuant [1] is that feature decorrelation before scalar quantization dramatically improves rate-distortion behavior. In particular, a random orthogonal transform spreads information across coordinates so that per-coordinate Lloyd-Max quantization becomes far more effective than quantizing the original basis directly.

The main drawback is cost. For a head dimension d, a dense orthogonal transform requires $O(d^2)$ parameters and arithmetic, which is difficult to justify in latency-sensitive settings such as autoregressive decoding. RotorQuant [2] addresses this issue by replacing the dense transform with sparse blockwise rotations in the geometric algebra $\mathrm{Cl}(3, 0)$. This preserves the qualitative idea of local decorrelation while reducing complexity to linear in d.

Despite this progress, the 3D block structure leaves performance on the table. First, it is not hardware aligned: common head dimensions such as 64, 128, and 256 are powers of two, so partitioning into triples creates awkward remainder handling and non-ideal memory layout. For example, $d = 128$ yields forty-two full 3D blocks plus one 2D tail. Second, a 3D rotation has only three intrinsic degrees of freedom, which limits how aggressively each local subspace can mix correlated coordinates.

This paper explores a different operating point. We move from 3D Clifford blocks to 4D quaternion blocks and leverage the Lie-theoretic decomposition

$$so(4) \cong su(2) \oplus su(2), \tag{1}$$

which implies that every 4D rotation can be represented by a pair of unit quaternions acting from the left and right. This gives a closed-form, low-overhead parameterization of $SO(4)$ that is both mathematically clean and implementation friendly. We therefore name the method **IsoQuant**, emphasizing the isoclinic structure that underlies its blockwise $SO(4)$ transform.

Contributions. We make four contributions.

1. We introduce **IsoQuant**, a 4D block rotation scheme for online vector quantization based on quaternion sandwich products.
2. We derive two practical instantiations: **IsoQuant-Full** with double-sided $SO(4)$ rotations, and **IsoQuant-Fast** with a single isoclinic factor for extreme efficiency.
3. We analyze why 4D blocks are especially attractive for systems deployment: they improve arithmetic efficiency, eliminate most boundary handling, and align naturally with SIMD and fused-kernel execution.
4. We provide a fused CUDA implementation and show in fair kernel-level comparisons against RotorQuant fused CUDA kernels that IsoQuant achieves consistent speedups while preserving reconstruction quality.

2 Related Work

Online vector quantization and KV-cache compression. TurboQuant [1] frames online vector quantization as a dense-rotation-plus-scalar-quantization pipeline and combines it with QJL residual correction [3]. RotorQuant [2] reduces the heavy dense transform by replacing it with sparse 3D Clifford rotors. Other KV-cache compression methods such as KIVI [4] and KVQuant [5] focus on calibration, asymmetric quantization, or hardware-aware packing rather than explicit geometric decorrelation.

Quaternion and geometric representations. Unit quaternions are a classical tool for representing rotations [6]. In machine learning, algebraically structured representations have appeared in equivariant models, geometric neural networks, and efficient transformations on structured features. RiemannFormer [7] provides a recent geometric treatment that explicitly discusses the isoclinic decomposition of $SO(4)$ in the setting of curved-space attention. Our use of this structure is narrower and more systems-oriented: we exploit the same $SO(4)$ factorization to design a cheap blockwise decorrelating transform for low-bit quantization.

3 Problem Setup

We consider the stage-1 mean-squared-error component of online vector quantization. Given an input vector $x \in \mathbb{R}^d$, we seek an encoder E, scalar quantizer Q, and decoder D such that

$$\hat{x} = D(Q(E(x))) \tag{2}$$

minimizes reconstruction error while keeping both parameter count and online compute small.
As in prior work [1, 2], we factor the problem into three pieces:

1. an orthogonal or approximately orthogonal transform that decorrelates coordinates;
2. coordinate-wise scalar quantization in the transformed basis;
3. an inverse transform to reconstruct the vector.

To stabilize the scalar quantizer, we additionally separate norm and direction. Writing

$$x = \rho\,\bar{x}, \qquad \rho = \|x\|_2, \qquad \|\bar{x}\|_2 = 1, \tag{3}$$

we quantize the normalized direction $\bar{x}$ and store or transmit the norm ρ separately. This follows the implementation pattern used by efficient quantizers and keeps the transformed coordinates within a predictable dynamic range.

4 Quaternion and $SO(4)$ Preliminaries

We identify each 4D block with a quaternion:

$$v = x_0 + x_1\mathbf{i} + x_2\mathbf{j} + x_3\mathbf{k} \in \mathbb{H}, \tag{4}$$

where $\mathbb{H}$ denotes the quaternion algebra and $\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1$ [6]. The quaternion conjugate of $q = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ is

$$\overline{q} = a - b\mathbf{i} - c\mathbf{j} - d\mathbf{k}. \tag{5}$$

Unit quaternions form the 3-sphere S^3 and provide a convenient representation of rotations. In 4D, the relevant structure is richer than the familiar $SO(3)$ case. The Lie algebra decomposition

$$so(4) \cong su(2) \oplus su(2) \tag{6}$$

implies that a general rotation in $SO(4)$ can be expressed using two independent unit quaternions. This interpretation is closely connected to the isoclinic decomposition of $SO(4)$, which is discussed in detail by RiemannFormer [7] in the broader context of attention over curved spaces. We build on the same decomposition, but use it for hardware-efficient local decorrelation in online quantization.

Proposition 1. *Let $q_L, q_R \in S^3$ be unit quaternions. The map*

$$T(v) = q_L \, v \, \overline{q_R} \tag{7}$$

defines an orthogonal transformation on $\mathbb{R}^4$. Its inverse is

$$T^{-1}(v) = \overline{q_L} \, v \, q_R. \tag{8}$$

Moreover, (q_L, q_R) and $(-q_L, -q_R)$ induce the same element of $SO(4)$.

The left factor corresponds to a left-isoclinic rotation and the right factor to a right-isoclinic rotation. Together they cover the full six degrees of freedom of $SO(4)$ without the singularities of Euler-angle parameterizations.

5 Method: IsoQuant

5.1 Blockwise 4D Rotation

Given $\bar{x} \in \mathbb{R}^d$, we partition it into

$$g = \left\lceil \frac{d}{4} \right\rceil \tag{9}$$

blocks,

$$\bar{x} = \left[v^{(1)}, v^{(2)}, \ldots, v^{(g)} \right], \tag{10}$$

where each $v^{(i)} \in \mathbb{R}^4$ is treated as a quaternion. If d is not divisible by 4, the final block is zero padded.
Each block is rotated independently, quantized coordinate-wise, and inverse rotated:

$$v^{(i)} \mapsto \tilde{v}^{(i)} \mapsto \hat{v}^{(i)} \mapsto v^{(i)}_{\text{rec}}. \tag{11}$$

The recovered blocks are concatenated and the original norm ρ is restored.

5.2 IsoQuant-Full

IsoQuant-Full uses the complete double-sided action of $SO(4)$. For block i, we maintain a pair of unit quaternions $(q_L^{(i)}, q_R^{(i)})$ and compute

$$\tilde{v}^{(i)} = q_L^{(i)} v^{(i)} \overline{q_R^{(i)}}, \tag{12}$$

$$\hat{v}^{(i)} = Q\left(\tilde{v}^{(i)}\right), \tag{13}$$

$$v^{(i)}_{\text{rec}} = \overline{q_L^{(i)}} \hat{v}^{(i)} q_R^{(i)}. \tag{14}$$

This uses the full six-dimensional rotational freedom of $SO(4)$ and offers the strongest local mixing.

5.3 IsoQuant-Fast

IsoQuant-Fast restricts the transform to a single isoclinic factor:

$$\tilde{v}^{(i)} = q_L^{(i)} v^{(i)}, \tag{15}$$

$$\hat{v}^{(i)} = Q\left(\tilde{v}^{(i)}\right), \tag{16}$$

$$v^{(i)}_{\text{rec}} = \overline{q_L^{(i)}} \hat{v}^{(i)}. \tag{17}$$

Geometrically, this corresponds to a 3-dimensional subgroup of $SO(4)$ isomorphic to $SO(3)$. It sacrifices expressivity for lower parameter count, lower arithmetic cost, and simpler kernels.

Table 1: Forward rotation complexity at $d = 128$.

Method	Block Structure	Params	FMAs
TurboQuant [1]	dense 128×128	16,384	16,384
RotorQuant [2]	$43 \times$ 3D blocks	172	$\approx$ 2,408
IsoQuant-Full	$32 \times$ 4D blocks	256	1,024
IsoQuant-Fast	$32 \times$ 4D blocks	128	512

5.4 Parameterization and Learning

To avoid constrained optimization on the manifold directly, we parameterize each unit quaternion by an unconstrained vector and normalize it on the fly:

$$q_L^{(i)} = \frac{u_L^{(i)}}{\|u_L^{(i)}\|_2}, \qquad q_R^{(i)} = \frac{u_R^{(i)}}{\|u_R^{(i)}\|_2}, \tag{18}$$

where $u_L^{(i)}, u_R^{(i)} \in \mathbb{R}^4$ are free parameters. This keeps optimization in Euclidean space while enforcing the unit-quaternion constraint implicitly in the computation graph. A practical lightweight variant is to sample the initial u vectors from a Gaussian distribution and keep them fixed, yielding random block rotations analogous to the randomized transform used by TurboQuant.

5.5 Quantization Pipeline

Algorithm 1 summarizes the stage-1 pipeline.

Algorithm 1 IsoQuant Stage-1 Quantization

Require: Input vector $x \in \mathbb{R}^d$, scalar quantizer Q, mode $\in \{\textsc{Full}, \textsc{Fast}\}$
Compute norm $\rho \leftarrow \|x\|_2$ and normalized vector $\bar{x} \leftarrow x / \max(\rho, \varepsilon)$
Partition $\bar{x}$ into $g = \lceil d/4 \rceil$ zero-padded 4D blocks
for $i = 1$ to g **do**
  if mode $=$ FULL **then**
    $\tilde{v}^{(i)} \leftarrow q_L^{(i)} v^{(i)} \overline{q_R^{(i)}}$
    $\hat{v}^{(i)} \leftarrow Q(\tilde{v}^{(i)})$
    $v_{\text{rec}}^{(i)} \leftarrow \overline{q_L^{(i)}} \hat{v}^{(i)} q_R^{(i)}$
  else
    $\tilde{v}^{(i)} \leftarrow q_L^{(i)} v^{(i)}$
    $\hat{v}^{(i)} \leftarrow Q(\tilde{v}^{(i)})$
    $v_{\text{rec}}^{(i)} \leftarrow \overline{q_L^{(i)}} \hat{v}^{(i)}$
  end if
end for
Concatenate reconstructed blocks and drop padded coordinates
return $\hat{x} \leftarrow \rho\,[v_{\text{rec}}^{(1)}, \ldots, v_{\text{rec}}^{(g)}]$

6 Complexity Analysis

A quaternion multiplication uses 16 scalar multiplications and 12 scalar additions. To match common systems reporting conventions, we count this as approximately 16 fused multiply-add operations. For $d = 128$, the forward rotation cost becomes straightforward to compare.

The comparison highlights two regimes. IsoQuant-Full uses somewhat more parameters than RotorQuant because each block stores two quaternions, but it still remains tiny relative to dense rotations and cuts rotation arithmetic by more than $2\times$. IsoQuant-Fast goes further and is cheaper than RotorQuant in both parameters and compute.

More generally, if $g = \lceil d/4 \rceil$, then the parameter count is $8g$ for Full and $4g$ for Fast. The forward rotation cost is $32g$ FMAs for Full and $16g$ FMAs for Fast. Both scale linearly in d.

7 Systems Considerations

7.1 Why 4D Blocks Are Hardware Friendly

The choice of block size is not purely algebraic. It directly shapes memory layout, vectorization efficiency, and kernel fusion opportunities.

Alignment. Most transformer head dimensions are powers of two. A 4D partition therefore avoids the pathological tails induced by 3D chunking in almost every common setting. At $d = 128$, IsoQuant uses exactly 32 blocks with no remainder, whereas a 3D design requires 42 full blocks plus a leftover fragment.

Vectorization. Four-wide blocks fit naturally into SIMD-friendly load and store patterns such as `float4`. This reduces boundary checks and helps both CPU SIMD backends and GPU kernels maintain regular control flow.

Kernel fusion. The per-block transform is a closed-form sequence of one or two quaternion products, coordinate-wise scalar quantization, and an inverse transform. This structure is especially suitable for fused kernels in online quantization pipelines because the entire block can often remain in registers from input load through output store.

7.2 Prototype Mapping

Our current prototype follows exactly this design pattern: it packs feature vectors into 4D blocks, applies either double-sided or single-sided quaternion rotation, performs scalar Lloyd-Max quantization, and reconstructs the result with a fused CUDA kernel. In addition, we built a benchmark harness that directly compares the fused IsoQuant kernels against the fused RotorQuant CUDA kernel under matched tensor shapes, bit widths, and data types.

8 Compatibility with Residual Correction

IsoQuant is intended as a replacement for the stage-1 decorrelation transform, not as a rejection of later residual correction. In two-stage pipelines such as TurboQuant [1], one can keep the residual inner-product correction unchanged:

$$r = x - \hat{x}_{\mathrm{mse}}. \tag{19}$$

The residual can still be projected with a quantized Johnson-Lindenstrauss transform [3] or a related low-bit correction mechanism. In this sense, IsoQuant is complementary to existing stage-2 estimators: it reduces the cost of the orthogonalization step while remaining compatible with unbiased inner-product correction.

9 Experimental Results

9.1 Research Questions

We aim to answer four questions:

1. Does a 4D quaternion block improve reconstruction quality over a 3D rotor block at fixed bit width?
2. Does IsoQuant-Full justify its extra parameters relative to IsoQuant-Fast?
3. How much wall-clock speedup does hardware alignment deliver in fused kernels?
4. Does improved local mixing translate into better downstream KV-cache quality for real LLM inference?

9.2 Current Scope

This version evaluates the stage-1 quantize–dequantize path, focusing on two metrics that are directly attributable to the decorrelation transform itself:

1. reconstruction MSE on normalized synthetic vectors; and
2. fused CUDA kernel latency for the stage-1 blockwise transform.

We leave full end-to-end KV-cache evaluation with residual correction and downstream attention metrics to future work. This separation is deliberate: in TurboQuant-style pipelines, stage-1 primarily affects reconstruction distortion and residual energy, while final inner-product fidelity also depends on the stage-2 estimator.

9.3 Baselines

We compare against three families of baselines:

1. **Dense rotation:** TurboQuant [1], as the conceptual dense-rotation reference for stage-1 complexity.
2. **Block rotation:** RotorQuant [2], as the primary method-level baseline.
3. **Fair kernel baseline:** the fused CUDA kernel in `rotor_fused_kernel.cu` from the RotorQuant codebase, directly benchmarked against our fused CUDA kernel.

9.4 Metrics

We report both algorithmic and systems metrics:

- reconstruction MSE on normalized synthetic vectors;
- parameter count and estimated arithmetic complexity;
- fused-kernel latency under matched tensor shapes;
- speedup relative to RotorQuant fused CUDA.

9.5 Setup

All experiments in this section use synthetic normalized vectors with batch size 8192. We evaluate all combinations of

$$d \in \{128, 256, 512\}, \qquad b \in \{2, 3, 4\}, \qquad \text{dtype} \in \{\texttt{fp16}, \texttt{fp32}\}, \tag{20}$$

for a total of 18 fused-kernel benchmark settings. We focus on these dimensions because they cover the most common per-head or grouped-KV widths in practical LLM deployments, while avoiding the large-dimension kernel-specialization regime that would otherwise dominate the discussion. On the IsoQuant side, we evaluate both IsoQuant-Full and IsoQuant-Fast. On the RotorQuant side, we report both the fused CUDA kernel and, in separate ablations, the higher-level PyTorch module path.

All fused CUDA benchmarks in this section were conducted on a single NVIDIA RTX 4090 GPU, and we report kernel latencies under the same hardware setting for both RotorQuant and IsoQuant.

9.6 Fused CUDA Head-to-Head Results

Table 2 reports representative fused CUDA results. The main takeaway is that IsoQuant is consistently faster than RotorQuant under an apples-to-apples kernel comparison while preserving essentially identical reconstruction quality. Across all 18 settings, IsoQuant-Full achieves an average speedup of 4.83× over RotorQuant fused CUDA, and IsoQuant-Fast achieves an average speedup of 5.08×. The strongest settings are low-bit and medium-width regimes, where speedups exceed 6× while MSE remains unchanged up to the reported precision.

The quality story is equally encouraging. In every tested setting, the MSE of IsoQuant is either indistinguishable from RotorQuant or slightly lower. For example, in the largest tested FP32 configuration ($d = 512, b = 4$), RotorQuant fused CUDA achieves MSE 1.8×10^{-5}, while IsoQuant-Fast matches this value and IsoQuant-Full remains within the same numerical range. We therefore do not observe a tradeoff in which IsoQuant wins speed by degrading reconstruction quality.

9.7 Aggregate Trends

The full sweep reveals three robust trends.

1. **IsoQuant-Fast is consistently the fastest variant.** It outperforms IsoQuant-Full in all tested settings, though the gap is modest compared with the larger gain over RotorQuant.
2. **The gain is strong in both FP16 and FP32.** Averaged over all tested FP16 settings, IsoQuant-Full and IsoQuant-Fast achieve 4.83× and 5.06× speedups, respectively; the corresponding FP32 averages are 4.84× and 5.10×.
3. **Low-bit and medium-width settings are especially favorable.** Several configurations in the ($d = 128, 256$) range exceed 6× speedup, showing that the compact 4D formulation amortizes per-block overhead particularly well in practical KV-cache regimes.

These trends are consistent with the systems argument developed earlier. Compared with RotorQuant's 3D Clifford blocks, IsoQuant avoids the expansion to an 8-component multivector representation, keeps the per-block state smaller, and aligns naturally with four-wide memory and register organization. The result is not a change in asymptotic complexity—both methods remain linear in d—but a real and repeatable reduction in constant factors.

Table 2: Fused CUDA comparisons against RotorQuant on normalized synthetic vectors with batch size 8192. Latencies are in μs.

dtype	bits	dim	RotorQuant	IsoQuant-Full	IsoQuant-Fast	Full speedup	Fast speedup
fp16	2	128	32.6	8.9	8.7	3.64	3.76
fp16	3	128	34.5	5.8	5.7	5.94	6.00
fp16	4	128	44.0	6.9	7.0	6.34	6.31
fp16	2	256	32.5	8.7	8.5	3.75	3.83
fp16	3	256	36.6	6.1	6.0	6.00	6.11
fp16	4	256	46.5	8.1	7.4	5.75	6.26
fp16	2	512	36.0	8.3	6.9	4.33	5.19
fp16	3	512	53.2	13.2	11.3	4.03	4.69
fp16	4	512	87.1	24.1	21.7	3.62	4.02
fp32	2	128	28.7	5.8	5.9	4.92	4.84
fp32	3	128	33.9	5.9	6.0	5.74	5.65
fp32	4	128	44.0	6.8	6.8	6.46	6.46
fp32	2	256	34.5	6.8	6.2	5.07	5.56
fp32	3	256	35.2	6.7	6.6	5.25	5.32
fp32	4	256	47.6	8.0	7.5	5.95	6.35
fp32	2	512	36.7	11.4	10.6	3.22	3.45
fp32	3	512	53.4	15.6	14.3	3.42	3.73
fp32	4	512	90.8	25.7	23.5	3.53	3.87

Table 3: Ablation axes. Completed settings in this work are marked by checkmarks.

Axis	Values	
Rotation type	Full, Fast	✓
Bit width	2, 3, 4 bits	✓
Block size	3D, 4D, optionally 8D grouped variants	
Quaternion parameters	random fixed ✓, learned normalized	
Residual correction	off ✓, QJL-style correction on	
Deployment backend	PyTorch ✓, fused CUDA	✓

9.8 Module-Level vs Kernel-Level Speedups

In addition to fused-kernel measurements, we also benchmarked higher-level PyTorch module paths. There, the apparent speedups can reach roughly 4×–10×, because the IsoQuant prototype currently enjoys a more streamlined execution path than the baseline RotorQuant module. However, we regard the fused CUDA comparison as the fairest measure of method-intrinsic systems advantage. The fused results therefore serve as our primary hardware claim, while the larger module-level gains should be interpreted as an implementation-dependent systems outcome.

9.9 Ablations

9.10 What Remains for a Full Submission

The current experiments establish a strong stage-1 result, but a full submission should still add downstream KV-cache metrics:

1. integration with a stage-2 residual correction module such as QJL;
2. attention-logit preservation and inner-product error under the complete two-stage pipeline;
3. retrieval and perplexity experiments on real KV tensors extracted from deployed LLMs.

These follow naturally from the present implementation because IsoQuant only replaces the stage-1 transform and remains compatible with the same residual correction machinery used by TurboQuant and RotorQuant.

10 Limitations and Future Work

IsoQuant is not a full solution by itself.

1. **Block locality.** Like other block-diagonal transforms, it does not mix information across blocks, so global correlations remain unaddressed.
2. **Evaluation gap.** This draft now includes fused CUDA benchmarks and reconstruction experiments, but the final conference version still needs end-to-end KV-cache validation on real models and tasks.
3. **Learning dynamics.** Although normalized quaternion parameters are simple to optimize, the relative value of learned versus random rotations remains an empirical question.
4. **Stage-2 interaction.** The best coupling between 4D block decorrelation and residual correction may depend on the downstream attention estimator.

Promising next steps include hierarchical cross-block mixing, jointly learned codebooks and quaternion parameters, and specialized kernels for fused KV-cache compression during autoregressive decoding.

11 Conclusion

IsoQuant replaces awkward 3D block rotations with a hardware-aligned 4D quaternion formulation grounded in the isoclinic decomposition of $SO(4)$. The resulting design retains the spirit of blockwise decorrelation while offering stronger local mixing, cleaner memory alignment, and lower arithmetic cost. Our fused CUDA experiments on practical dimensions $d \in \{128, 256, 512\}$ show that this design advantage is measurable in practice: across 18 matched settings, IsoQuant-Full and IsoQuant-Fast deliver average speedups of 4.83× and 5.08× over RotorQuant fused CUDA while preserving essentially identical reconstruction MSE. These results make a strong case that quaternion-based 4D blocks are a compelling stage-1 replacement for online vector quantization, and they motivate the next step of integrating the method with residual correction and full KV-cache evaluation.

References

[1] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni. TurboQuant: Online vector quantization with near-optimal distortion rate. In *International Conference on Learning Representations*, 2026.

[2] J. D. Pope. RotorQuant: Clifford algebra vector quantization for LLM KV cache compression. 2026. https://www.scrya.com/rotorquant/. Code: https://github.com/scrya-com/rotorquant.

[3] A. Zandieh, M. Daliri, and I. Han. QJL: 1-bit quantized JL transform for KV cache quantization with zero overhead. *arXiv preprint arXiv:2406.03482*, 2024.

[4] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *arXiv preprint arXiv:2402.02750*, 2024.

[5] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.

[6] J. B. Kuipers. *Quaternions and Rotation Sequences*. Princeton University Press, 1999.

[7] Z. Ji. RiemannFormer: A framework for attention in curved spaces. *arXiv preprint arXiv:2506.07405*, 2025.